

## Contents

|                                                        |    |
|--------------------------------------------------------|----|
| 1. Core OOP Principles (Pillars of OOP) .....          | 3  |
| 1. Class .....                                         | 4  |
| 2. Object.....                                         | 5  |
| 3. Abstraction .....                                   | 6  |
| 4. Encapsulation.....                                  | 7  |
| 5. Inheritance.....                                    | 8  |
| 6. Polymorphism.....                                   | 9  |
| Compile-time Polymorphism (Method Overloading) .....   | 9  |
| Runtime Polymorphism (Method Overriding).....          | 10 |
| 2. Java-Specific Implementation of OOP .....           | 11 |
| 3. Inheritance in Java .....                           | 11 |
| 4. Polymorphism in Java .....                          | 11 |
| 5. Abstraction in Java.....                            | 12 |
| 6. Encapsulation in Java .....                         | 12 |
| 7. Advanced OOP Features .....                         | 12 |
| 8. OOP Design Concepts (For Deeper Understanding)..... | 12 |
| Questions.....                                         | 13 |
| 1. Questions on Class.....                             | 13 |
| Basic Conceptual Questions on Java Class.....          | 13 |
| Class Members (Fields and Methods) .....               | 13 |
| Types and Modifiers .....                              | 13 |
| Class vs Interface .....                               | 13 |
| Code-based & Practical Questions .....                 | 13 |
| Memory and Design .....                                | 14 |
| Advanced .....                                         | 14 |
| 2. Questions on Object .....                           | 15 |
| Basic Conceptual Questions .....                       | 15 |
| Syntax and Usage.....                                  | 15 |
| Object Behavior & Life Cycle .....                     | 15 |
| Code-based / Practical Questions.....                  | 15 |
| Advanced Object Usage.....                             | 16 |
| Other Questions .....                                  | 17 |



## 1. Core OOP Principles (Pillars of OOP)

- Class
- Object
- Abstraction
- Encapsulation
- Inheritance
- Polymorphism
  - Compile-time (Method Overloading)
  - Runtime (Method Overriding)

## 1. Class

Definition: A class is a **blueprint or template** for creating objects. It defines the **properties (fields)** and **behaviors (methods)** that the objects created from the class will have.

Key Points:

- Defines **structure and behavior**
- Can have fields, methods, constructors, and blocks
- No memory allocated until an object is created

```
class Car {  
  
    String color;  
  
    int speed;  
  
    void drive() {  
        System.out.println("Car is driving");  
    }  
}
```

## 2. Object

Definition: An object is an **instance of a class**. It occupies memory and represents a real-world entity.

We create objects in Java to **bring classes to life**, access their members, and build structured, modular, and maintainable programs. Without objects, Java would not support **object-oriented programming**.

Key Points:

- Created using `new` keyword
- Has state (fields) and behavior (methods)
- You can create multiple objects from one class

```
Car myCar = new Car();  
myCar.color = "Red";  
myCar.drive();
```

### 3. Abstraction

Definition: Abstraction means **hiding the internal implementation** and showing only the **essential features** to the user.

```
abstract class Animal {  
    abstract void sound(); // Abstract method  
    abstract void eat(); // Abstract method  
}  
  
class Dog extends Animal {  
    void sound() {  
        System.out.println("Bark");  
    }  
    void eat(){  
        System.out.println("Meat");  
    }  
}
```

Key Points:

- Achieved using **abstract classes** and **interfaces**
- Helps manage complexity
- Promotes loose coupling

## 4. Encapsulation

Definition: Encapsulation is the **wrapping of data (variables)** and **code (methods)** together into a **single unit (class)**. It also restricts direct access to data using **access modifiers**.

```
class Person {  
    private String name; // Private variable  
  
    public String getName() { // Public getter  
        return name;  
    }  
  
    public void setName(String name) { // Public setter  
        this.name = name;  
    }  
}
```

Key Points:

- Achieved using **private variables** and **public getters/setters**
- Provides **data hiding**
- Makes the code more secure and maintainable

## 5. Inheritance

Definition: Inheritance is the process where one class **acquires the properties and methods** of another class.

```
class Animal {
    void eat() {
        System.out.println("This animal eats food.");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Dog barks.");
    }
}

class Monkey extends Animal {
    void jump() {
        System.out.println("Monkey Jumps.");
    }
}
```

Key Points:

- `extends` keyword is used
- Promotes **code reuse**
- Java supports **single**, **multilevel**, and **hierarchical** inheritance (but not multiple inheritance with classes)



## 6. Polymorphism

Definition: Polymorphism means "**many forms**". It allows methods to behave **differently** based on the object that invokes them.

### Compile-time Polymorphism (Method Overloading)

Definition: Same method name, but different **parameter lists** in the same class.

```
class MathUtils {  
    int add(int a, int b) {  
        return a + b;  
    }  
  
    double add(double a, double b) {  
        return a + b;  
    }  
}
```

Key Points:

- Resolved at **compile time**
- Improves readability
- Also called **static polymorphism**

## Runtime Polymorphism (Method Overriding)

Definition: A **subclass provides a specific implementation** of a method that is already defined in its superclass.

```
class Animal {  
    void sound() {  
        System.out.println("Animal makes a sound");  
    }  
}
```

```
class Cat extends Animal {  
    @Override  
    void sound() {  
        System.out.println("Meow");  
    }  
}
```

```
Animal a = new Cat(); // Runtime polymorphism  
a.sound(); // Output: Meow
```

Key Points:

- Uses **method overriding**
- Achieved via **inheritance**
- Resolved at **runtime**
- Enables **dynamic method dispatch**

## 2. Java-Specific Implementation of OOP

- Classes and Objects in Java
  - Constructors (Default, Parameterized, Copy)
  - `this` keyword
  - `new` keyword
  - `static` keyword (with variables, methods, blocks)
  - `final` keyword (variable, method, class)
  - Access Modifiers (`private`, `public`, `protected`, `default`)
- 

## 3. Inheritance in Java

- Single Inheritance
  - Multilevel Inheritance
  - Hierarchical Inheritance
  - Why Java doesn't support Multiple Inheritance with classes
  - `super` keyword (constructor and method usage)
- 

## 4. Polymorphism in Java

- **Compile-time (Static Polymorphism)**
    - Method Overloading
  - **Runtime (Dynamic Polymorphism)**
    - Method Overriding
    - Dynamic Method Dispatch
    - `instanceof` operator
-

## 5. Abstraction in Java

- Abstract Classes and Methods
  - Interfaces
    - Default methods (Java 8+)
    - Static methods
    - Functional interfaces (Java 8+)
    - Difference between abstract class vs interface
- 

## 6. Encapsulation in Java

- Getters and Setters
  - JavaBeans convention
  - Data hiding and access control
- 

## 7. Advanced OOP Features

- Nested Classes (Static and Non-static inner classes)
  - Anonymous Classes
  - Lambda Expressions (Java 8+)
  - Object class methods (`toString()`, `equals()`, `hashCode()`, `clone()`)
  - `enum` in Java
  - Record Classes (Java 14+)
- 

## 8. OOP Design Concepts (For Deeper Understanding)

- Association, Aggregation, Composition
- Coupling and Cohesion
- SOLID Principles (for writing better OOP code)
- DRY, KISS, YAGNI (coding principles)
- Design Patterns (basic):
  - Singleton, Factory, Builder, etc.

## Questions

### 1. Questions on Class

#### Basic Conceptual Questions on Java Class

1. What is a class in Java?
2. How is a class different from an object?
3. Why do we use classes in Java?
4. What is the syntax of declaring a class in Java?
5. What is the default access modifier for a class?
6. Can a Java class be declared inside another class? What is it called?
7. What is the difference between a class and a structure (struct in C/C++)?

#### Class Members (Fields and Methods)

8. What are class variables and instance variables?
9. What are static and non-static members in a class?
10. How are methods defined inside a class?
11. What is a constructor? How is it different from a method?
12. Can a class have multiple constructors? What is constructor overloading?

#### Types and Modifiers

13. What are different types of classes in Java? (e.g., regular class, abstract class, final class)
14. What is the use of the `final` keyword with a class?
15. What is an abstract class? Can we create an object of it?
16. What is a nested class? List its types.
17. Can a class be both `abstract` and `final`? Why or why not?

#### Class vs Interface

18. What is the difference between a class and an interface?
19. Can a class implement multiple interfaces?
20. Can a class extend more than one class? Why?

#### Code-based & Practical Questions

21. Write a simple Java class with fields, constructors, and methods.

22. Write a program that creates objects of a class and calls its methods.
23. Demonstrate the use of static variables and static methods inside a class.
24. Write a program that uses constructor overloading in a class.
25. Create a class `Student` with fields `name`, `rollNo`, and `marks` and print student details.

## Memory and Design

26. How is a class loaded into memory in Java?
27. What is a class loader?
28. What is the difference between a class and an object in terms of memory?

## Advanced

29. What is reflection? How can we get class information at runtime using reflection?
30. What is a singleton class? How do you implement it in Java?
31. What is a POJO (Plain Old Java Object)?
32. What is the difference between an inner class and a static nested class?

## 2. Questions on Object

Definition: An object is an **instance of a class**. It occupies memory and represents a real-world entity.

We create objects in Java to **bring classes to life**, access their members, and build structured, modular, and maintainable programs. Without objects, Java would not support **object-oriented programming**.

Key Points:

- Created using `new` keyword
- Has state (fields) and behavior (methods)
- You can create multiple objects from one class

```
Car myCar = new Car();  
myCar.color = "Red";  
myCar.drive();
```

### Basic Conceptual Questions

1. What is an object in Java?
2. How is an object different from a class?
3. What are the steps involved in creating an object in Java?
4. Can you create an object without using the `new` keyword? If yes, how?
5. What happens in memory when you create an object?
6. Why can't we call non-static methods without creating an object?
7. What is the default value of an object reference in Java?

### Syntax and Usage

8. How do you create an object from a class in Java?
9. What is object instantiation?
10. Can we have multiple objects of the same class?
11. Can an object access both static and non-static members?
12. What is the purpose of constructors in object creation?

### Object Behavior & Life Cycle

13. What is object reference?
14. What is the scope of an object?
15. When is an object eligible for garbage collection?
16. What is the difference between object creation and object initialization?
17. What happens if we try to use an object reference that is null?

### Code-based / Practical Questions

18. Write a program to demonstrate creating multiple objects of a class.
19. Write a program to access class variables and methods through objects.
20. Demonstrate the difference between static and non-static members using objects.
21. Write a program to pass an object as a method parameter.
22. Write a program that returns an object from a method.

## Advanced Object Usage

23. What is object cloning? How do you clone an object in Java?
24. What is the difference between shallow copy and deep copy of an object?
25. What is the use of the `this` keyword with objects?
26. What are anonymous objects? Give an example.
27. How does Java handle object comparison (using `==` vs `.equals()`)?



## Other Questions

Why Do We Need to Create an Object in Java?

- A **class** is like a blueprint (e.g., blueprint of a car).
- An **object** is the actual **instance** created from that blueprint (a real car built from the plan).

You need to create objects to:

- Access **non-static** methods and variables.
- Represent **real-world entities** like a `Student`, `BankAccount`, or `Car`.
- Enable **modularity**, **reusability**, and **data encapsulation**.

```
class Car {
    String color = "Red";
    void drive() {
        System.out.println("The car is driving.");
    }
}

public class Main {
    public static void main(String[] args) {
        Car myCar = new Car(); // object created
        System.out.println(myCar.color); // access variable
        myCar.drive(); // call method
    }
}
```

Advantages of Creating Objects in Java

| Advantage           | Description                                                                                               |
|---------------------|-----------------------------------------------------------------------------------------------------------|
| Encapsulation       | Objects allow grouping data (variables) and behavior (methods) together, hiding internal details.         |
| Reusability         | Classes can be reused by creating multiple objects without rewriting code.                                |
| Modularity          | Code is easier to manage because each object handles its own data and functions.                          |
| Flexibility         | Objects make it easier to modify one part of the program without affecting others.                        |
| Real-world modeling | Objects let you model real-world entities in code more naturally.                                         |
| Inheritance         | You can create new classes based on existing ones, reusing and extending functionality.                   |
| Polymorphism        | Objects of different classes can be treated as objects of a common superclass, allowing dynamic behavior. |

